ADR-0001: Algoritmo de limite de dicas por jogador

Architecture Decision Record

Status	Proposto
Data	2026-10-02
Escopo	Serviço de recomendação de cartas (RateLimitService)

1. Contexto

O jogador pode solicitar **dicas** ao sistema de recomendação, geradas pelo motor **Minimax**. É necessário limitar a quantidade de dicas que cada jogador pode utilizar dentro de determinado período.

Esse limite representa uma **cota de gameplay**, sendo uma regra de design do jogo e não um mecanismo de proteção da capacidade da infraestrutura. O objetivo principal é controlar o impacto das dicas na experiência e no equilíbrio do jogo.

A **janela de utilização** (diária ou semanal) e o **limite de dicas (N)** ainda não foram definidos. Essas decisões dependem das regras do sistema de batalha, que ainda estão sendo elaboradas.

Por exemplo, dependendo da duração e da quantidade de batalhas, uma regra de 5 dicas por dia pode ser adequada ou excessiva, enquanto 5 dicas por semana pode ser insuficiente.

Portanto, o algoritmo escolhido deve funcionar independentemente da janela definida, mantendo tanto o período quanto o limite como **configurações**, e não como regras fixas no código.

2. Direcionadores da decisão

#	Direcionador	Peso
D1	Clareza para o jogador: "restam X dicas, renovam em..."	Alto
D2	Janela e limite configuráveis sem trocar de algoritmo	Alto
D3	Simplicidade de implementação e teste	Alto
D4	Atomicidade: dois pedidos simultâneos não podem ultrapassar a cota	Alto
D5	Baixo custo de estado por jogador	Médio
D6	Tolerância a rajadas / suavização de tráfego	Baixo, pois não é o objetivo

3. Opções consideradas

3.1 Fixed Window Counter

Mantém um contador de dicas utilizadas por jogador dentro de uma janela fixa, como um dia ou uma semana.

Ao iniciar uma nova janela, a cota é renovada integralmente.

Exemplo:

```
Limite: 5 dicas por dia

Segunda-feira
──────────────────────────────
Dica 1 ✓
Dica 2 ✓
Dica 3 ✓
Dica 4 ✓
Dica 5 ✓
Dica 6 ✗
──────────────────────────────
Limite atingido

Terça-feira
──────────────────────────────
Contador reiniciado
Dica 1 ✓
...
```

Prós:

- Implementação simples;
- Estado mínimo;
- Semântica diretamente relacionada à regra de gameplay;
- Fácil exibição na interface;
- Fácil cálculo de quando a cota será renovada;
- Funciona tanto para janelas diárias quanto semanais.

Contras:

- Permite utilizar a cota no final de uma janela e novamente no início da próxima, podendo resultar em até `2N` dicas em um intervalo curto.

Esse comportamento é considerado aceitável porque corresponde à semântica de uma cota que é renovada em um período fixo.

3.2 Sliding Window

Considera as dicas utilizadas durante um intervalo móvel de duração `T`.

Por exemplo, com limite de 5 dicas em 24 horas, o sistema verifica quantas dicas foram utilizadas nas últimas 24 horas, independentemente da mudança do dia.

Prós:

- Evita o efeito de fronteira do Fixed Window;
- O limite é respeitado em qualquer intervalo de duração `T`.

Contras:

- Maior complexidade;
- Pode exigir o armazenamento dos horários das utilizações;
- A explicação para o jogador é menos direta;
- Torna mais complexo informar quando exatamente uma nova dica estará disponível;
- Não apresenta vantagem relevante para a regra de gameplay atual.

3.3 Token Bucket

Utiliza um conjunto de tokens que são consumidos a cada dica e reabastecidos continuamente a uma determinada taxa.

Prós:

- Permite controlar rajadas;
- Permite regeneração gradual das dicas;
- Possui baixo custo de estado.

Contras:

- A semântica é diferente de uma cota fixa por dia ou semana;
- Exige configuração de taxa de reposição e capacidade;
- Adiciona complexidade desnecessária caso as dicas sejam renovadas integralmente;
- Seria mais adequado caso o design definisse algo como "1 dica recuperada a cada 4 horas".

3.4 Leaky Bucket

Utiliza uma fila e controla a taxa na qual as requisições são processadas.

Prós:

- Mantém uma taxa de processamento previsível;
- Útil para suavização de tráfego.

Contras:

- Não representa diretamente uma cota de dicas;
- Introduz o conceito de fila;
- Não há necessidade de enfileirar solicitações de recomendação para o objetivo atual;
- É mais adequado para controle de fluxo do que para regras de gameplay.

4. Comparativo

Critério	Fixed Window	Sliding Window	Token Bucket	Leaky Bucket
Semântica de "X por dia/semana"	Exata	Aproximada	Não	Não
Clareza na UI	Alta	Média	Média	Baixa
Trocar dia por semana	Somente configuração	Somente configuração	Requer ajuste de parâmetros	Requer ajuste de parâmetros
Estado por jogador	1 contador	O(N) ou 2 contadores	2 valores	Fila/estado
Complexidade	Muito baixa	Média	Média	Média/alta
Efeito de fronteira	Sim, aceitável	Não	Não	Não
Regeneração gradual	Não	Não	Sim	Não
Adequação ao problema atual	Alta	Média	Média	Baixa

5. Decisão

> **Adotar o Fixed Window Counter** para controlar a quantidade de dicas disponíveis por jogador.

O tamanho da janela e o limite de dicas serão mantidos como **parâmetros de configuração**, permitindo que o sistema seja configurado posteriormente como, por exemplo:

```
hints:
  window: day
  limit: 5
```

ou:

```
hints:
  window: week
  limit: 20
```

A escolha do Fixed Window está diretamente relacionada à forma como a funcionalidade é apresentada ao jogador: uma quantidade definida de dicas disponível durante um período definido.

Além disso, o algoritmo apresenta menor complexidade de implementação e teste, possui baixo custo de estado e permite alterar a regra de gameplay sem modificar a estratégia de controle.

O efeito de fronteira do Fixed Window é considerado aceitável neste cenário, pois a renovação da cota representa uma decisão explícita de design do jogo.

6. Questões em aberto

#	Questão	Depende de
Q1	A janela será diária, semanal ou ambas?	Sistema de batalha
Q2	Qual será o valor de N?	Duração e quantidade de batalhas
Q3	Qual será o fuso utilizado para o reset?	Público-alvo do jogo
Q4	Caso seja semanal, qual será o início da semana?	Q1
Q5	Uma dica cuja geração falhar consumirá a cota?	Regra de negócio
Q6	Solicitar uma dica para o mesmo estado de batalha novamente consumirá uma nova dica?	Sistema de batalha

Essas questões não impedem a escolha do algoritmo, pois tanto a janela quanto o limite poderão ser configurados posteriormente.

7. Consequências

Positivas

- Regra simples de explicar;
- Implementação simples;
- Facilidade de testes;
- Baixo custo de armazenamento;

- Fácil alteração entre limite diário e semanal;
- Fácil exibição de dicas restantes;
- Fácil cálculo do momento de renovação.

Negativas / Riscos

- O jogador pode concentrar o uso da cota na transição entre duas janelas;
- Não há regeneração gradual das dicas;
- Caso o design do jogo passe a utilizar recuperação contínua, o algoritmo precisará ser reavaliado.

8. Gatilhos para revisão

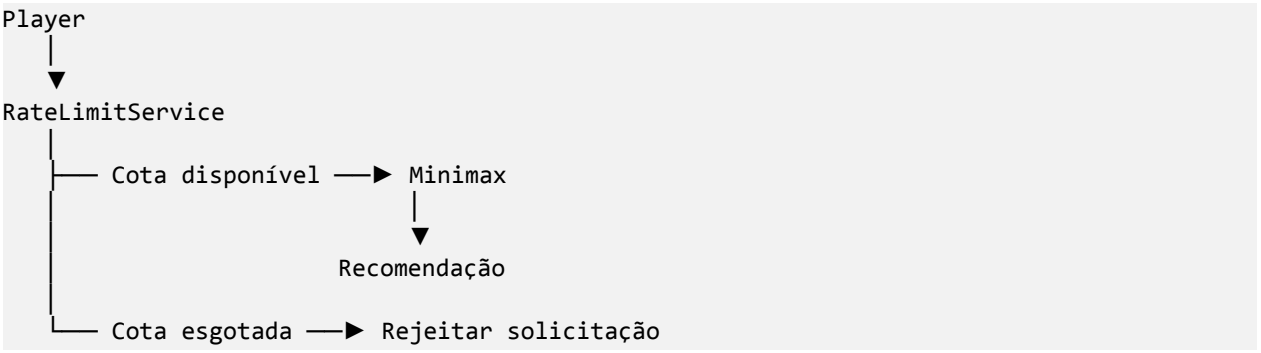
Este ADR deverá ser revisado caso:

- O jogo passe a utilizar **regeneração gradual** de dicas: avaliar Token Bucket;
- O requisito passe a ser "no máximo N dicas em qualquer intervalo de T": avaliar Sliding Window;
- O endpoint também precise de proteção contra abuso técnico: adicionar um Rate Limiter independente para infraestrutura, sem substituir a cota de gameplay.

9. Notas de implementação

A implementação deverá manter a lógica de Rate Limiting separada da lógica de geração da recomendação.

Conceitualmente:



A implementação poderá utilizar PostgreSQL, Redis ou outra solução de armazenamento definida pela arquitetura do sistema. A escolha do armazenamento não faz parte deste ADR.

O controle deverá ser **atômico**, garantindo que duas solicitações simultâneas do mesmo jogador não consigam ultrapassar a cota disponível.

Ao atingir o limite, a API poderá informar:

- quantidade de dicas restantes;
- limite total;
- momento da próxima renovação.

Isso permite que a interface apresente uma informação semelhante a:

> **Você possui 2 dicas restantes. A cota será renovada amanhã às 00:00.**

10. Referências

- RFC 6585: Status Code 429 Too Many Requests
- Documentação do PostgreSQL: INSERT ... ON CONFLICT
- Documentação do PostgreSQL: date_trunc
- Referências sobre algoritmos de Rate Limiting: Fixed Window, Sliding Window, Token Bucket e Leaky Bucket